Week 7 (Days 43-49): Explaining models, engineering features, imbalance, pipelines, clustering

Status: written in full. Titles match `SYLLABUS.md`.

Week goals

1. Explain a model honestly: impurity vs permutation importance, and exact Shapley values computed by brute force.
2. Turn raw columns (hours, categories, text) into features a model can use, and *measure* which encoding helps.
3. Handle rare positives with the right metrics, class weights and a threshold chosen on validation data.
4. Make leakage impossible with `Pipeline` + `ColumnTransformer`, and prove the leak exists when you don't.
5. Begin unsupervised learning: k-means from scratch, then hierarchical clustering, DBSCAN and Gaussian mixtures via EM.
6. Segment real wholesale customers and validate the segments without labels.

```
flowchart LR
  I[Importance & Shapley] --> F[Feature engineering]
  F --> IM[Imbalance & thresholds]
  IM --> P[Leak-proof pipelines]
  P --> K[k-means]
  K --> C[DBSCAN / hierarchical / GMM-EM]
  C --> S[Project: customer segments]
```

New datasets: bike sharing (hourly rentals in Washington DC, 2011-2012, 17,379 rows, OpenML id 42712) and **wholesale customers** (440 clients of a Portuguese distributor, annual spend in 6 categories, UCI). Both are fetched by `common.py` loaders; URLs are in `data/README.md`.

Day 43: Feature importance, permutation importance, SHAP intuition

Review today: Day 42 (+1), Day 40 (+3), Day 36 (+7), Day 22 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why explain? To debug (is the model using a leaked ID?), to build trust, to generate hypotheses, and sometimes because regulation requires it. Remember, though, that importance describes **the model**, not the world. A feature can matter to the model without causing the outcome.

Three tools. 1. **Impurity importance (MDI):** for trees, sum how much each feature's splits reduced impurity. It is free and fast. It is computed on *training* data and is known to favour high-cardinality or continuous features, which have more possible split points. 2. **Permutation importance:** shuffle one column in held-out data and measure the drop in score. It is model-agnostic and measured on test data. Caveat: with two correlated features, shuffling one leaves the other to carry the signal, so both look unimportant. 3. **Shapley values (the idea behind SHAP):** a fair split of one prediction among features, borrowed from game theory. Feature j's value is its marginal contribution v(S ? {j}) - v(S), averaged over all orders in which features could "join". It has three key properties: - **Efficiency:** the values sum to f(x) - f(baseline). - **Symmetry:** equal contributors get equal shares. - **Null player:** a feature that never changes anything gets 0.

Exact computation needs 2? subsets. Libraries approximate it, or use model structure (TreeSHAP).

A sanity trick: add a column of pure random numbers. Any feature ranked *below* it is indistinguishable from noise.

Block 2: Worked example (45 min)

`code/week-07/day43_importance.py` on California, with a random forest (100 trees, test R^2 0.7921) plus a planted `RANDOM_id` column:

feature	MDI	permutation (test R^2 drop)
MedInc	0.5277	0.7917

feature	MDI	permutation (test R^2 drop)
Latitude	0.0811	0.3938
Longitude	0.0807	0.2979
AveOccup	0.1336	0.1993
HouseAge	0.0551	0.0696
AveRooms	0.0456	0.0304
AveBedrms	0.0285	0.0081
Population	0.0288	0.0078
RANDOM_id	0.0189	0.0021

What to notice: 1. The two methods disagree on the ordering. MDI ranks AveOccup second, while permutation ranks Latitude and Longitude well above it. Location matters a lot on held-out data. 2. **Honest finding:** in this run the random column ranked **last (9 of 9)** even by MDI. The well-documented MDI bias towards continuous noise features did *not* show up strongly here (min_samples_leaf=1 forests on 16k rows). Its MDI of 0.0189 is close to Population's 0.0288, though, so Population and AveBedrms are barely above noise under either method.

Exact Shapley by brute force (a linear model on 4 features, baseline = training mean, test row 0 with MedInc 4.152, HouseAge 22, AveRooms 5.663, Latitude 32.58): ? = MedInc +0.122, HouseAge -0.1147, AveRooms -0.0043, Latitude +0.1348. They sum to **0.1379**, which equals f(x) - f(baseline) (efficiency). They also match the known closed form for linear models, w?(x? - mean?), exactly. That gives you a test that your Shapley code is right before you trust it on a non-linear model.

Block 3: Coding lab (2h)

1. Implement permutation importance yourself (5 shuffles per feature, report mean +/- std) and compare with `sklearn.inspection.permutation_importance`.
2. Correlation trap: add `Latitude_copy = Latitude + small noise` and recompute both importances. What happens to Latitude's permutation importance?
3. Extend the brute-force Shapley code to the random forest on 6 features (64 subsets per feature) for 3 rows. Check that efficiency holds.
4. Write a paragraph explaining one prediction to a non-technical reader, using your Shapley values.
5. Stretch: install `shap` in your own venv (it is not part of the course requirements) and compare `TreeExplainer` values with your brute force on the 6-feature forest.

Block 4: Practice problems (1h)

1. Two features, f = 3.x? + 2.x?, baseline (0, 0), x = (1, 1). Shapley values? *(3 and 2.)*
2. f = x?.x? with baseline (0, 0) and x = (1, 1). Shapley values? *(0.5 each: v({1}) = v({2}) = 0 and v({1,2}) = 1.)*
3. Why compute permutation importance on test rather than train data? *(Train importance rewards memorised features.)*
4. Two perfectly correlated features. What does permutation show? *(Both low; the model uses either.)*
5. Does high importance imply causation? *(No.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Name the three importance methods and one flaw of each.
- State Shapley efficiency.
- What does the random-column trick test?
- Which feature topped both lists?
- Why did Latitude rise under permutation?

Going deeper (optional)

"Baseline" choice changes Shapley values. The interventional version (ours) replaces missing features with baseline values and can create unrealistic rows, such as a big house with zero rooms. Conditional versions sample realistic values but blur credit between correlated features. No version is "the truth". Say which one you used.

Common mistakes / how to debug

- Reporting MDI on an overfit forest as "what matters".
- Explaining a model whose test score is poor. Explanations of a bad model explain nothing about the world.

What to do next

Tomorrow you *create* features, and measure that a good encoding can matter more than the choice of model.

Day 44: Feature engineering: encodings, dates, text counts

Review today: Day 43 (+1), Day 41 (+3), Day 37 (+7), Day 23 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. A model sees only numbers. How you present a concept decides what is easy to learn. "Hour = 17" as a number tells a linear model that demand rises steadily from midnight to 23:00, which is false. As 24 one-hot columns, each hour gets its own level. As (sin, cos) of 2pi.hour/24, hour 23 sits next to hour 0.

A toolkit. - **Categoricals:** - one-hot (linear models; watch cardinality), - ordinal (trees), - target encoding (category -> mean target, which *must* be cross-fitted to avoid leakage), - native categorical support (HistGB). - **Dates and times:** hour, weekday, month, holiday flags, time since an event, cyclic encodings. - **Interactions:** hour x workingday (rush hour exists only on working days). Linear models need these explicitly. Trees find them. - **Numeric:** log for heavy tails, ratios (bedrooms/rooms), binning for thresholds. - **Text:** bag-of-words counts, **TF-IDF** (down-weight words that appear everywhere: idf = ln((1 + N)/(1 + df)) + 1 in scikit-learn's smoothed form), and n-grams.

Process. Hypothesise, encode, then *measure with the same CV* before keeping a feature.

Block 2: Worked example (45 min)

`code/week-07/day44_feature_eng.py` on bike sharing (17,379 hourly rows, target = rentals per hour). Rows are in time order, so we use `TimeSeriesSplit(5)`.

Hourly mean demand shows two peaks, at **08:00 (359)** and **17:00 (461)**, with a trough of 6 at 04:00. It is a commute pattern no straight line in "hour" can follow.

model (RidgeCV unless noted)	CV MAE (rentals/hour)
predict the training mean	148.3
no hour feature	123.8
hour as a number	114.8
hour cyclic (sin, cos)	101.4
hour one-hot	87.2
hour one-hot x workingday	**72.6**
HistGB on raw features (native categoricals)	**51.3**

Lessons: the encoding alone moved the linear model from 114.8 to 72.6, a 37% cut. Cyclic encoding helps, but only a little, because two smooth waves can't produce two sharp peaks. The interaction term captured "rush hours exist only on working days". The tree ensemble found all of this by itself and still won by a wide margin, which tells you the remaining signal is non-linear (weather x hour, and so on).

Text counts on 4 toy reviews: the vocabulary has 11 words. "battery" appears in 3 of 4 documents, so its idf is **1.223**. "cracked" appears in 1, so its idf is **1.916**. Rare words carry more weight.

A fix we needed: OpenML stores this dataset's `holiday` and `workingday` as categorical "True"/"False" strings, so the script converts them with `astype(str) == "True"`. Always check dtypes after loading.

Block 3: Coding lab (2h)

1. Reproduce the table. Then add `weekday` one-hot and `hour x weekday` (168 columns). Does it beat 72.6?
2. Add a "temperature band" (binned temp) x hour interaction. Measure it.
3. Implement **cross-fitted target encoding** for `hour` yourself (out-of-fold means), and compare with naive in-sample target encoding. Report the CV MAE of both. The naive one should look suspiciously good in-sample.
4. Text: on the 4 reviews, compute TF-IDF by hand for one document using the formula above, and compare with `TfidfVectorizer`. (Note that it also L2-normalises each row.)
5. Stretch: `sklearn.preprocessing.SplineTransformer` on hour (with `extrapolation="periodic"`). Can a smooth periodic spline beat one-hot?

Block 4: Practice problems (1h)

1. sin and cos encoding of hour 6 and hour 18? *(sin 1, cos 0; sin -1, cos 0.)*
2. Why does one-hot of a 100k-level ID column hurt? *(Huge sparse width, overfitting, unseen IDs.)*
3. Why must target encoding be cross-fitted? *(Otherwise each row's own label leaks into its feature.)*
4. A word appears in every document. What is its sklearn smoothed idf? *(ln(1) + 1 = 1, its minimum.)*
5. Which hour encoding did best for ridge, and why? *(One-hot x workingday: arbitrary level per hour, separately for working days.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- List four categorical encodings and when to use each.
- Why is "hour as a number" bad for linear models?
- Write the TF-IDF idea in one sentence.
- Why use TimeSeriesSplit here?
- How much did encoding improve ridge MAE?

Going deeper (optional)

Feature hashing (`HashingVectorizer`, `FeatureHasher`) maps unbounded vocabularies into a fixed number of columns via a hash function. Collisions are rare enough to be tolerable, and you need no vocabulary in memory. This is how large-scale linear models handle billions of categorical values, and it previews the embedding tables of Month 3.

Common mistakes / how to debug

- Computing a feature with the full dataset (global mean demand per hour) outside CV, which leaks.
- Shuffled CV on time-ordered data.

What to do next

Tomorrow: when positives are rare, accuracy lies. You will learn the metrics and the threshold dial.

Day 45: Imbalanced data: class weights, resampling, threshold tuning

Review today: Day 44 (+1), Day 42 (+3), Day 38 (+7), Day 24 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The accuracy trap. With 2% fraud, "never fraud" scores 98% accuracy and catches nothing. On imbalanced problems, look at: - **Precision:** of those flagged, how many are real? - **Recall:** of the real ones, how many did we flag? - **F1:** their harmonic mean. - **PR-AUC (average precision):** ranking quality focused on the positive class. Its no-skill baseline is the positive rate, not 0.5. - **ROC-AUC:** ranking quality overall. It can look comfortable even when precision is terrible, because the huge negative class makes the false-positive *rate* small.

Three levers. 1. **Class weights:** multiply the loss of positive rows (`class_weight="balanced"` uses weights inversely proportional to class frequency). This shifts the probabilities upwards. 2. **Resampling:** random oversampling of positives, or undersampling of negatives (SMOTE-style synthesis is a variant). Resample **only the training folds**, never validation or test. 3. **Threshold tuning:**

keep the model and move the decision threshold away from 0.5. Choose it on a **validation** set to meet a business target (best F1, recall >= 0.8, or a cost ratio).

For ranking quality (AUC), weights and resampling mostly don't help. They mainly move where 0.5 falls, which threshold tuning does directly and transparently.

Block 2: Worked example (45 min)

`code/week-07/day45_imbalanced.py` uses a synthetic task (20,000 rows, 20 features, about 2% positive; 60/20/20 train/validation/test split, so 88 positives in test). Verified test results:

approach	acc	prec	rec	F1	PR-AUC	ROC-AUC
always negative	0.9780	0.000	0.000	0.000	0.022	0.500
logreg, threshold 0.5	0.9788	1.000	0.034	0.066	0.235	0.715
logreg, class_weight=balanced	0.7180	0.046	0.602	0.086	0.150	0.725
logreg, random oversampling	0.7157	0.046	0.602	0.085	0.145	0.723
logreg, threshold 0.0756 (tuned on validation)	0.9485	0.157	0.307	**0.208**	0.235	0.715

How to read it: 1. Plain logistic regression "beats" the trivial model on accuracy by 0.0008, catching 3 of 88 positives. 2. Balanced weights and oversampling behave almost identically (they are nearly equivalent for this model). They trade precision for recall, and at their implied 0.5 threshold F1 barely moves. 3. Picking the threshold on validation data for best F1 (0.0756) gave the best F1 on test, **0.208**, without retraining. 4. A business rule "recall >= 0.8" needs threshold 0.0105, at precision **0.032**: 30 false alarms per true catch. That is a model-quality problem, not a threshold problem. ROC-AUC 0.715 is weak.

Block 3: Coding lab (2h)

1. Reproduce the table. Plot the precision-recall curve and mark the 0.5, 0.0756 and 0.0105 points.
2. Write `best_threshold(y_val, p_val, metric)` for F1, and for minimum expected cost with cost(FN) = 20.cost(FP).
3. Prove the resampling leak: oversample *before* splitting, and compare validation F1 with the correct version.
4. Try `HistGradientBoostingClassifier` on the same data. Does better ranking (PR-AUC) help more than any rebalancing?
5. Stretch: apply all of this to Day 42's adult data, picking a threshold for precision >= 0.7. (The capstone on Day 58 does exactly this.)

Block 4: Practice problems (1h)

1. 1000 rows, 20 positive. The model flags 50, of which 15 are positive. Precision, recall and F1? *(0.30, 0.75, 0.4286.)*
2. What is the no-skill PR-AUC for a 2% positive rate? *(About 0.02.)*
3. Why choose the threshold on validation, not test? *(Otherwise test stops being an unbiased estimate.)*
4. When is class weighting preferable to threshold moving? *(When the training loss itself should focus on positives, e.g. for trees whose splits then change. Also when the model has no probabilistic output.)*
5. Is ROC-AUC 0.715 with PR-AUC 0.235 good or bad at a 2% base rate? *(PR-AUC is about 10x baseline, but precision at useful recall is still low.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Why is accuracy misleading here?
- Define precision and recall.
- What does class weighting change?
- Where must resampling happen?
- What threshold and F1 did validation tuning give?

Going deeper (optional)

With costs c_FP and c_FN and calibrated probabilities, the Bayes-optimal threshold is t = c_FP/(c_FP + c_FN). For FN 20x worse than FP, t = 1/21 ~= 0.048. This only works if the probabilities are calibrated (Day 54), which class weighting breaks. Prefer an unweighted model plus a threshold, or recalibrate.

Common mistakes / how to debug

- Reporting ROC-AUC alone on rare-event problems.
- Resampling before CV splits.
- A threshold tuned on test data.

What to do next

Tomorrow: pipelines that make all these leaks structurally impossible.

Day 46: Pipelines, ColumnTransformer, leakage-proof preprocessing

Review today: Day 45 (+1), Day 43 (+3), Day 39 (+7), Day 25 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Leakage means information from validation or test rows sneaks into training. It inflates scores and breaks in production. The classic forms are: - Preprocessing (scaling, imputation, feature selection, target encoding) fitted on all rows before splitting. - Duplicates across splits. - Features computed from the future. - Features that are proxies for the label.

The structural fix: a `Pipeline`. Chain transformers and a final estimator into one object. When `cross_validate` refits it per fold, every `fit` sees only that fold's training rows. `ColumnTransformer` applies different preprocessing to different columns: numeric -> impute + scale, categorical -> impute + one-hot. `make_column_selector(dtype_include=...)` picks columns by type.

Searching inside pipelines. Hyperparameters are addressed as `step__param` (e.g. `clf__C`, `prep__cat__onehot__min_frequency`). So `GridSearchCV` can tune preprocessing and model together.

Deployment bonus. One fitted object maps raw rows to predictions. Persist it on Day 57.

Block 2: Worked example (45 min)

`code/week-07/day46_pipelines.py`. Verified:

1. The leak, demonstrated on pure noise. 200 rows, 5000 random features and random labels, so the truth is 50% accuracy. - Select the 20 features most correlated with y using *all rows*, then 5-fold CV a logistic regression: **0.770 accuracy**. That is pure fiction. - Put `SelectKBest` *inside* the pipeline: **0.470**, i.e. chance, which is the truth.

2. Adult with a full pipeline. Numeric: median impute + scale. Categorical: constant impute "missing" + one-hot with `handle_unknown="ignore"`. Then logistic regression. A grid over C gives CV ROC-AUC 0.01: 0.9048, 0.1: 0.9068, 1: 0.9068, 10: 0.9067. Best C = 0.1, **test ROC-AUC 0.9057** (touched once). - The fitted transformer produces **108 features**, with readable names (`num__age`, `cat__native-country_missing`, ...) from `get_feature_names_out()`. - A row with occupation = "Astronaut" (never seen in training) still gets a prediction (0.016) instead of a crash. Unknown categories become all-zeros.

Block 3: Coding lab (2h)

1. Rebuild the adult pipeline from memory with `make_column_selector`. Print the step names, and set one parameter with `set_params`.
2. Reproduce the noise leak with k = 5, 20 and 100 selected features. How does the fake accuracy change with k?
3. Write a custom transformer class (`fit` returns self, `transform` adds `log1p(capital-gain)` and a `capital-gain > 0` flag), using `BaseEstimator` and `TransformerMixin`. Put it in the pipeline and grid-search a flag that turns it on or off.
4. Tune `OneHotEncoder(min_frequency=...)` in {1, 20, 100} together with C. How many features does each produce?
5. Stretch: draw a pipeline diagram (Jupyter renders one; in a script, `sklearn.utils.estimator_html_repr` writes HTML).

Block 4: Practice problems (1h)

1. Which of these leak if done before the split: (a) dropping a column, (b) standard scaling, (c) target encoding, (d) removing exact duplicate rows? *(b mildly; c badly. a is safe if it doesn't depend on y. d is safe and recommended before splitting.)*
2. Parameter name for C in `Pipeline([("prep", ct), ("clf", LogisticRegression())])`? *(`clf__c`.)*
3. Why did noise-feature selection give 0.77? *(With 5000 features, some correlate with y by chance on these 200 rows, and selection saw the validation rows.)*
4. What does `handle_unknown="ignore"` do? *(It encodes unseen categories as all-zeros.)*
5. Name two leaks a pipeline does *not* prevent. *(Future-information features; duplicates across splits.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Define leakage with two examples.
- How does a pipeline prevent preprocessing leakage?
- What were the leaky and honest accuracies on noise?
- How do you name nested parameters?
- What happens with unseen categories?

Going deeper (optional)

Kaufman et al.'s taxonomy of leakage (2012, "Leakage in data mining") distinguishes leakage in features (proxies for the target) from leakage in training examples (rows that cross the split boundary). Search for it if you want the formal treatment. Its core lesson is that leakage is usually found by suspicion of *too-good* results.

Interview-style question (15 min, written)

"Our model scored 0.99 AUC in the notebook and 0.71 in production. What happened?" Write a debugging plan in five steps. 1. Diff the training and serving code paths. Was preprocessing fitted on all data, or re-implemented by hand at serving time? 2. List every feature's timestamp against the prediction time, to spot future information. 3. Search for duplicate or near-duplicate entities across the splits. 4. Rank features by importance and ask whether the top one is a proxy for the label. 5. Rebuild the evaluation with one `Pipeline` and a split that mimics production (by time or by entity).

Connect each step to a leak type from Block 1.

Common mistakes / how to debug

- `fit_transform` on test data. Only `transform`.
- Forgetting that `GridSearchCV.best_estimator_` was already refit on the full training split.

What to do next

Tomorrow begins unsupervised learning: there are no labels, only structure.

Day 47: k-means clustering and choosing k

Review today: Day 46 (+1), Day 44 (+3), Day 40 (+7), Day 26 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Drop k pins on a map of points. Each point joins its nearest pin. Then move each pin to the centre of its group. Repeat until nothing moves. That is **Lloyd's algorithm** for k-means. It minimises **inertia**, the total squared distance from points to their centroid:

J = Sum_i ||x_i - mu_{c(i)}||^2

Each step (assign, then update) can only lower J, so it converges, but only to a **local** minimum that depends on the starting pins.

Initialisation. Random starts can put two pins in one blob and none in another. **k-means++** picks each new initial centroid with probability ? squared distance to the nearest existing centroid, so pins spread out. scikit-learn uses it by default with several

restarts (`n_init`), keeping the lowest inertia.

Choosing k. Inertia always falls as k grows, so you can't minimise it over k. Two heuristics: - **Elbow:** look for the k where the drop flattens. - **Silhouette:** for each point, s = (b - a)/max(a, b), where a is its mean distance to its own cluster and b is its mean distance to the nearest other cluster. The average lies in [-1, 1], and higher is better.

Neither is truth. Ask what the clusters are *for*.

Assumptions. k-means likes round, similar-sized, similar-density clusters, and it needs scaled features (Euclidean distance again).

Block 2: Worked example (45 min)

`code/week-07/day47_kmeans.py`. Verified: - **Blobs** (1500 points, 4 true clusters with std 1.0): scratch Lloyd reaches inertia 2681.1 in 9 iterations. scikit-learn reaches 2680.9. ARI against the true labels is 0.8303 (Adjusted Rand Index: 1 is perfect, 0 is chance). - **Initialisation matters.** Over 20 seeds, random init ranged from inertia **2680.9 to 20954.7**. k-means++ ranged from 2680.9 to 3530.5. - **Choosing k:**

k	inertia	silhouette
2	21344.5	0.7136
3	3754.6	**0.7821**
4	2680.9	0.6083
5	2370.1	0.5822
6	2141.2	0.4453
7	1890.6	0.3371
8	1689.6	0.3263

Honest finding: silhouette picks **k = 3**, but the data was generated with 4 clusters. Two of the true centres are only 2.08 apart (with cluster std 1.0), so they overlap, and silhouette prefers merging them into one well-separated group. The elbow (a big drop to k = 3, a smaller one to 4, then a flat curve) is ambiguous between 3 and 4. This is normal. Validation without labels is a judgement call. - **Digits**, k = 10 on raw pixels: 43 iterations in about 35 ms. ARI against the true digit is 0.6512, and purity is 0.7891. The majority-digit map [2, 8, 4, 3, 1, 5, 1, 6, 0, 7] shows that **digit 1 owns two clusters and digit 9 owns none**. 9s merge with similar-looking digits.

Block 3: Coding lab (2h)

1. Write `kmeans(X, k, iters, seed)` with vectorised assignment (the ||x||^2 - 2x.mu + ||mu||^2 trick) and empty-cluster handling (re-seed the empty cluster at the farthest point). Assert that inertia never increases.
2. Implement k-means++ initialisation and reproduce the random vs ++ ranges.
3. Plot inertia and silhouette against k. Then compute the **gap statistic**-style comparison: the inertia of a uniform-random dataset of the same bounding box, at each k.
4. Digits: show each centroid as an 8x8 image. Do they look like digits?
5. Stretch: Mini-batch k-means on 200,000 points (`MiniBatchKMeans`). Compare its time and inertia with full k-means. (Week 8's project writes the assignment kernel in Rust.)

Block 4: Practice problems (1h)

1. Points 0, 2, 10, 12 (1-D), k = 2, initial centroids 0 and 2. Run Lloyd by hand. *(Assign {0}, {2, 10, 12} -> centroids 0 and 8; assign {0, 2}, {10, 12} -> 1 and 11; then stable.)*
2. Why can't you choose k by minimising inertia? *(It keeps falling until k = m.)*
3. Silhouette of a point with a = 1, b = 4? *(0.75.)*
4. Two features, income in rupees and age in years, unscaled. Which one drives the clusters? *(Income.)*
5. Complexity of one Lloyd iteration? *(O(m.k.n).)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**

- Describe Lloyd's two steps.
- Why does k-means converge, and only locally?
- What does k-means++ do?
- Define silhouette.
- Which k did silhouette pick, and what was the truth?

Going deeper (optional)

k-means is a hard-assignment, equal-spherical-variance limit of the Gaussian mixture fitted by EM (tomorrow). Shrink the GMM's shared variance to 0 and its soft responsibilities become hard nearest-centroid assignments. That is the bridge to probabilistic clustering.

Common mistakes / how to debug

- Using ARI to *choose* k when you have no labels. You don't, in real life.
- Clustering one-hot columns with Euclidean k-means. Consider k-modes or Gower distance.

What to do next

Tomorrow: clusters that are not round (DBSCAN, linkage) and soft, probabilistic clusters (GMM via EM).

Day 48: Hierarchical clustering, DBSCAN, Gaussian mixtures (EM intuition)

Review today: Day 47 (+1), Day 45 (+3), Day 41 (+7), Day 27 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Agglomerative (hierarchical) clustering. Start with every point as its own cluster and repeatedly merge the two closest clusters. "Closest" is the **linkage**: - **single:** nearest pair. It follows chains and handles odd shapes, but one bridge point merges two groups. - **complete:** farthest pair. - **average:** mean distance. - **ward:** the merge that increases inertia least. It gives k-means-like round clusters.

The merge history is a **dendrogram**. Cut it at a height to get clusters.

DBSCAN. A point is a **core** point if at least `min_samples` points lie within radius `eps`. Clusters are connected regions of core points, plus their border points, and everything else is **noise** (label -1). It finds arbitrary shapes and the number of clusters, but it needs `eps` tuned to the density, and it struggles when densities vary.

Gaussian mixture models (GMM) and EM. Assume the data comes from k Gaussians with weights pi?, means mu? and covariances Sum?. We don't know which Gaussian produced each point, so we use **Expectation-Maximisation**: - **E-step:** compute responsibilities r_i? = P(component j | x_i), soft memberships that sum to 1. - **M-step:** re-estimate pi?, mu? and Sum? as responsibility-weighted averages.

Each round cannot decrease the data log-likelihood. **BIC** = -2.log L + p.ln m trades fit against the number of parameters p, so lower is better. It is a principled way to choose k.

Block 2: Worked example (45 min)

`code/week-07/day48_clustering.py`. Verified:

Two moons (non-convex), ARI against the truth:

method	ARI
k-means k=2	0.2657
ward	0.3200
single linkage	**1.0000**
DBSCAN eps=0.2, min_samples=5	**1.0000**

DBSCAN's eps sensitivity: eps 0.05 -> 37 clusters with 183 noise points. 0.1 -> 3 clusters, 2 noise. 0.3 -> 2 clusters, 0 noise. 0.5 -> 1 cluster (everything merges).

EM from scratch on 3 Gaussian blobs: log-likelihood -5992.68 (iteration 1) -> -3266.95 (5) -> -3043.75 (20), converging at iteration 52. The average log-likelihood is -3.3819 for our EM and -3.3830 for scikit-learn's `GaussianMixture`, so ours found a marginally *better* local optimum. Means: ours [[-0.01, 0.03], [3.12, 3.01], [-0.04, 4.0]]. scikit-learn's [[-0.01, 0.03], [3.26, 2.99], [0.02, 3.97]]. The two overlapping components settled slightly differently.

BIC by components: 1: 7291.0, 2: 6387.6, **3: 6205.1**, 4: 6232.4, 5: 6277.9, 6: 6318.1. BIC correctly picks 3.

Soft assignment: the point (1.5, 1.5) gets responsibilities [0.032, 0.005, 0.963]. That is a hedge k-means can't express.

Block 3: Coding lab (2h)

1. Implement EM for a GMM with full covariances: E-step with `scipy.stats.multivariate_normal.logpdf` and log-sum-exp normalisation, M-step with weighted means and covariances plus 1e-6.I for stability. Assert that the log-likelihood is non-decreasing.
2. Plot the 2sigma ellipses of the fitted components over the data.
3. DBSCAN: plot the sorted distance to each point's 5th neighbour (the "k-distance plot") and choose eps at the knee. Does it agree with 0.2?
4. Draw a dendrogram (`scipy.cluster.hierarchy.dendrogram`) for 50 moons points with single and with ward linkage.
5. Stretch: three blobs with very different densities. Show that one eps can't fit all of them, then try `sklearn.cluster.HDBSCAN`.

Block 4: Practice problems (1h)

1. Why does single linkage solve moons while ward fails? *(Single follows chains of nearby points. Ward prefers compact round groups.)*
2. Which method labels outliers explicitly? *(DBSCAN, as -1.)*
3. Two equal-weight 1-D components N(0, 1) and N(4, 1); x = 2. Responsibilities? *(0.5 each, by symmetry.)*
4. Why can EM get stuck? *(It climbs to a local maximum; use multiple inits.)*
5. Parameter count of a 3-component, 2-D full-covariance GMM? *(Weights 2 + means 6 + covariances 3.3 = 17.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Name four linkages.
- Define a DBSCAN core point.
- What do the E- and M-steps compute?
- What does BIC trade off?
- Which methods solved moons?

Going deeper (optional)

EM is coordinate ascent on a lower bound of the log-likelihood (the evidence lower bound, ELBO). The E-step makes the bound tight, and the M-step maximises it. The same ELBO reappears in variational autoencoders in Month 3. Recognising it there will save you a day.

Interview-style question (15 min, written)

"Which clustering algorithm should we use for customer segments?" Write an answer that does not name a single winner. Ask about the shape of the clusters (k-means assumes compact convex blobs; DBSCAN handles arbitrary shapes and outliers), whether soft membership is wanted (a GMM gives probabilities), whether a hierarchy is useful to the business (dendrogram cuts at several levels), and the data size (agglomerative clustering is quadratic in memory). Then propose a protocol: scale the features, try two or three methods, compare their stability across bootstrap resamples, and validate the segments with the people who will use them. Day 47's silhouette result (k = 3 chosen when the truth was 4) shows that internal scores alone can mislead.

Common mistakes / how to debug

- Singular covariance (a component collapses onto one point, and the likelihood goes to infinity). Add a regulariser to the diagonal.
- Running DBSCAN on unscaled data, where eps means different things per feature.

What to do next

Tomorrow's project segments real wholesale customers, then checks whether the segments mean anything.

Day 49: PROJECT W7: customer segmentation; Quiz 7

Review today: Day 48 (+1), Day 46 (+3), Day 42 (+7), Day 28 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson: clustering for a business question (1h15)

Framing first. "Segment our customers" is not a question. "Find 2-5 groups whose buying patterns differ enough to justify different sales approaches" is. Good segments are: - **Distinct:** separated in feature space. - **Stable:** they reappear with other seeds or subsamples. - **Interpretable:** describable in a sentence. - **Actionable.**

The dataset: wholesale customers (UCI). 440 clients of a Portuguese wholesale distributor with annual spend on Fresh, Milk, Grocery, Frozen, Detergents_Paper and Delicassen, plus `Channel` (Horeca = hotel/restaurant/café, or Retail) and `Region`. We hold out `Channel` as an **external check**. It is never used in fitting.

Workflow. 1. Inspect distributions (spend is heavily right-skewed). 2. Transform (log), then scale. 3. Run k-means for a range of k and compare silhouette. 4. Profile each segment with medians. 5. Run stability checks. 6. Compare with the external label. 7. Cross-check with a GMM and BIC. 8. Visualise with PCA (Day 50 covers PCA properly; here it is just a 2-D view).

Block 2: Worked example: reference results (45 min)

`code/week-07/day49_week_project.py`. Verified: - **Skew:** Delicassen 11.15, Frozen 5.91, Milk 4.05. The means are far above the medians (Fresh: mean 12,000 vs median 8,504; max 112,151). - **Silhouette:**

k	log-scaled	raw-scaled
2	**0.2903**	0.5673
3	0.2592	0.3339
4	0.1885	0.3465

The *raw* silhouette looks much better (0.57), but it is driven by a few huge customers that sit far from everyone else, and those separate "nicely". After the log transform the structure is weaker but reflects typical customers. A higher silhouette is not automatically a better segmentation. - **Profiles (k=2, log-scaled), median annual spend:** - Segment 0 (n=252): Fresh 10,348, Grocery 2,406, Detergents 328. Fresh-food-heavy buyers. - Segment 1 (n=188): Grocery 11,527, Milk 7,690, Detergents 4,495. Shelf-goods buyers. - **External check against Channel:** segment 0 is 244 Horeca and 8 Retail. Segment 1 is 54 Horeca and 134 Retail. ARI is **0.5135**. The unsupervised segments rediscovered most of the restaurant/retail split without ever seeing it. - **Stability:** across 10 single-init runs with different seeds, the minimum pairwise ARI was **0.9909**. - **GMM BIC:** 1: 6596.3, 2: 6434.4, **3: 6269.0**, 4: 6312.4, 5: 6410.6. BIC prefers 3 components while silhouette prefers 2 k-means clusters. The methods disagree because they measure different things (likelihood under flexible ellipses vs separation of round groups). Report both, and choose by usefulness. - **PCA 2-D view** explains 0.4408 + 0.2719 = 71% of the log-scaled variance.

Block 3: Coding lab: the project (2h)

Deliverable: `notes/week07_segments.md` with figures. 1. **Explore (20 min):** histograms raw vs log, and a correlation matrix. Which spend categories move together? (Grocery and Detergents_Paper is a known strong pair. Verify it.) 2. **Cluster (30 min):** k-means for k = 2...7 on log-scaled data. Report silhouette and inertia, and choose k with a written argument. 3. **Profile (20 min):** a median table per segment and one-line personas. 4. **Validate (25 min):** a stability ARI over 10 seeds and over 80% subsamples. Then crosstab and ARI against Channel. 5. **Alternative (15 min):** a GMM with BIC for 1-6 components. Does the 3-component solution split one of your k-means segments? Which one? 6. **Recommend (10 min):** one paragraph for the sales team.

Rubric (self-grade out of 20):

criterion	points
Transform justified by the skew numbers	3
k chosen with >= 2 criteria plus a business argument	4
Segment profiles in plain language	3
Stability measured (seeds *and* subsamples)	3
External label used only for validation	3
Limitations stated (440 rows, one country, annual spend only)	2
Figures labelled and readable	2

Stretch: remove the 5 largest-spend customers and re-run. How much do the segments change? (This tests the outlier argument above.)

Block 4: Practice: Quiz 7 (1h), closed book, answers below

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Why log-transform spend?
- Why was raw silhouette misleading?
- What ARI did the segments reach against Channel?
- What did BIC prefer?
- Give one actionable recommendation.

Going deeper (optional)

Look up RFM (recency, frequency, monetary) segmentation, a standard retail framing. Our data only has the "monetary" part. Write down which additional data you'd request to build RFM, and why that data would change the segments.

Common mistakes / how to debug

- Interpreting segments on scaled values. Report medians in original units.
- Choosing k by the prettiest plot.

What to do next

Week 8: PCA and visualisation, anomaly detection, recommenders, time series, calibration, and the strategy for "what to try next", ending with a Rust k-means kernel.

Weekly quiz: Week 7

1. Name one flaw each of MDI and permutation importance.
2. State Shapley efficiency, and give our row-0 sum.
3. Shapley values for f = x?x?, baseline (0, 0), x = (1, 1)?
4. Which hour encoding gave ridge its best bike MAE, and what was it?
5. Why must target encoding be cross-fitted?
6. At 2% positives, what does "always negative" score on accuracy and PR-AUC?
7. What threshold did validation tuning pick on Day 45, and what F1 did it give?
8. On pure noise, what were the leaky and honest CV accuracies on Day 46?
9. What is k-means++ for, and what inertia range did random init give?
10. Which k did silhouette pick on the 4-blob data?
11. What does the EM E-step compute?
12. What ARI did the wholesale segments reach against Channel?

Answers

1. MDI is computed on training data and biased towards many-split-point features. Permutation importance splits credit badly between correlated features.
2. The values sum to f(x) - f(baseline). Row 0's sum was 0.1379.
3. 0.5 and 0.5.
4. One-hot x workingday, MAE 72.6.
5. Otherwise each row's own label leaks into its feature.
6. Accuracy 0.978. PR-AUC about 0.022 (the base rate).
7. 0.0756, giving F1 0.208.
8. 0.770 (leaky) vs 0.470 (honest).
9. Spreading initial centroids out. Random init ranged from 2680.9 to 20954.7.
10. k = 3 (the truth was 4).
11. Responsibilities P(component | x_i).
12. 0.5135.